

Guía Maestra: Arquitectura de Microservicios con Spring Cloud

Esta guía detalla la construcción y despliegue de un sistema de ventas desacoplado, diseñado para enseñar los fundamentos de la comunicación distribuida.

1. Configuración de Infraestructura

Mapa de Ruta (Spring Initializr)

Todos los proyectos deben configurarse con **Java 17/21** y **Maven**.

Proyecto	Dependencias Clave	Puerto
eureka-server	Eureka Server	8761
ms-stock	Web, JPA, MySQL, Eureka Client	8081
ms-pedidos	Web, JPA, MySQL, Eureka Client, OpenFeign	8082
api-gateway	Gateway (Reactive), Eureka Client	8080

Preparación de Bases de Datos (SQL)

Cada servicio es dueño de su propio esquema para garantizar el desacoplamiento.

SQL

```
CREATE DATABASE IF NOT EXISTS db_stock;
```

```
CREATE DATABASE IF NOT EXISTS db_pedidos;
```

```
-- Ejecutar en db_stock tras levantar los servicios:
```

```
USE db_stock;
```

```
INSERT INTO stock (producto_id, cantidad) VALUES (101, 50), (102, 10), (103, 0);
```

2. Estructura de los Componentes

A. Servidor de Descubrimiento (eureka-server)

Es el "directorío telefónico" del sistema. Sin él, los servicios no pueden encontrarse.

- **Anotación:** @EnableEurekaServer en la clase principal.
- **Propiedad Crítica:**

Properties

eureka.client.register-with-eureka=false

eureka.client.fetch-registry=false

B. API Gateway (api-gateway)

Punto de entrada único. Gestiona la seguridad y el enrutamiento.

- **Configuración (application.yml):** Define rutas hacia lb://ms-stock y lb://ms-pedidos.

C. Microservicio de Stock (ms-stock)

Gestiona la existencia física. Posee los paquetes: model, repository, service y controller.

- **Endpoint Clave:** GET /api/stock/validar/{id}.

D. Microservicio de Pedidos (ms-pedidos)

Orquesta la venta. **Punto clave:** La carpeta client.

- **Carpeta client:** Contiene la interfaz **Feign**. Es el puente que permite a Pedidos "hablar" con Stock sin conocer su IP.
- **Interfaz Feign:**

Java

```
@FeignClient(name = "ms-stock")
```

```
public interface StockClient {
```

```
    @GetMapping("/api/stock/validar/{productid}")
```

```
    boolean verificarStock(@PathVariable Long productid);
```

```
}
```

3. Guía de Despliegue (El "Ritual" de Inicio)

Para que el ecosistema funcione, el orden de encendido es obligatorio:

1. **Eureka Server (8761):** Debe estar activo para recibir registros.
 2. **Microservicios (Stock y Pedidos):** Se registran ante Eureka al iniciar.
 3. **API Gateway (8080):** Al final, para que al cargar ya pueda mapear los servicios registrados.
-

4. Pruebas de Integración (Postman)

Regla de Oro: Todas las consultas se hacen al puerto del Gateway (**8080**).

- **Validar Stock (GET):** `http://localhost:8080/api/stock/validar/101`
- **Crear Pedido (POST):** `http://localhost:8080/api/pedidos`
 - **Body (JSON):**

JSON

```
{  
  "productoid": 101,  
  "cantidad": 5  
}
```

5. Desafíos y Reflexión para el Alumno 🧠

🔧 Desafío de Desarrollo

1. **Validación Robusta:** Modificar el flujo para que el pedido falle si la cantidad solicitada es mayor al stock disponible (actualmente solo valida existencia).
2. **Sincronización:** Implementar que, tras un pedido exitoso, se descuente automáticamente la cantidad en ms-stock.

Debate de Resiliencia

- **Concurrencia:** ¿Qué ocurre si dos clientes compran el último ítem al mismo tiempo? (Concepto: @Transactional).
- **Fallas en Red:** ¿Qué sucede si el servicio de Stock se cae a mitad de una transacción? (Conceptos: **Circuit Breaker** y **Patrón Saga/Compensaciones**).

Nota para el alumno: Este diseño permite que cada servicio crezca de forma independiente. Si el servicio de pedidos tiene mucha demanda, podemos duplicarlo sin afectar al servicio de stock.